TCP-Based Multi-Client LAN Messaging: A Survey of Transport, Framing, Concurrency and Reliability, and the Case for a Hardened, Dependency-Free Reference Design

1st Harish Goud B
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
harishgoudbit@cbit.ac.in

2nd Revanth Reddy Lingala
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
revanthreddy0906@gmail.com

3rd Saarthak Manocha
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
saarthakmanocha10@gmail.com

***Abstract*—A local-area chat application looks trivial from the user's side, yet it forces a designer to solve four separate networking problems at once: recovering message boundaries from a byte stream, serving many clients concurrently, distributing each message to a group, and surviving abnormal disconnections. The research literature that addresses these problems is large but fragmented: transport standards describe the byte stream, server-architecture research compares threads with events, messaging standards such as IRC, XMPP, MQTT and WebSocket each fix one design point, and reliable-group-communication work studies guarantees far stronger than a classroom system needs. This paper surveys that literature through the lens of a concrete case, a command-line chat system written only with the Python standard library, in which one TCP server accepts many clients, runs one thread per connection, broadcasts UTF-8 messages, and frames every message with a four-byte length prefix. Comparing the reviewed works across framing, topology, concurrency, security and persistence shows a recurring pattern: each source treats one concern in isolation, and none presents a small, dependency-free artifact in which all of them can be inspected, tested and measured together. We argue that this is the missing piece and describe a hardened reference design that closes it through bounded outbound queues, heartbeats, an authenticated TLS channel and an asynchronous variant for controlled comparison. No performance figure is claimed; a measurement plan is given instead.**

***Index Terms*—TCP, socket programming, LAN chat, message framing, length prefix, concurrent server, thread-per-client, group messaging, reliability**

I. Introduction

A chat program rarely needs to do anything clever to be useful. Two people type, and each sees what the other typed. Underneath that simple behaviour, however, sits a stack of decisions that every networked application has to make. The server must accept connections and remember who is connected. It must turn a continuous stream of bytes back into the individual messages that were sent. It must handle many clients at the same time without letting shared state become corrupted. It must deliver each message to everyone else in the room, and it must notice and clean up after a client that vanishes without saying goodbye.

TCP is the natural transport, because it offers a reliable, ordered, full-duplex byte stream [2], [1]. The important word is *byte*. The current TCP specification deliberately exposes a stream rather than a sequence of records, so a single `send()` on one side does not correspond to a single `recv()` on the other [2], [7]. A message can arrive split across several reads, and several messages can arrive glued together in one read. Any chat protocol built directly on sockets therefore has to define its own message boundaries.

Several research and standards communities have worked on parts of this problem, each from a different angle. One community specifies the transport itself: reliability, flow control and congestion control [2], [4], [5]. A second community compares ways of serving many connections, from threads and processes to event loops and staged pipelines [11], [12], [14], [13], [17]. A third community defines complete messaging protocols, among them IRC, XMPP, MQTT and WebSocket [18], [20], [23], [22]. A fourth studies what it takes to give a group of processes consistent delivery in the presence of failures [26], [27], [25]. Finally, work on secure messaging explains what an eavesdropper or an impersonator can do to a plaintext channel and what the remedies cost [28], [29], [31].

What stands out when these bodies of work are placed side by side is how little they meet. A protocol standard tells a reader what a frame looks like, but not how a small server should schedule its handlers. A server-architecture paper compares throughput, but usually on web workloads rather than on a long-lived, many-to-many chat session. A group-communication paper reasons about failure models that a teaching system never approaches. Textbooks do walk through sockets and threads [7], [8], [9], but they present them as separate exercises, and a reader who wants to see framing, concurrency, broadcast and shutdown interacting in one short program has to assemble that picture alone.

This is the gap that motivates the paper. Rather than

choosing one of these threads to specialise in, we survey them together, anchored to one concrete artifact: a Python chat system, built with nothing beyond the standard library [32], [33], that we use as a running case. The contributions are:

1) a thematic survey of the transport, framing, concurrency, messaging-protocol, reliability and security literature relevant to a small TCP chat system;
2) a comparative analysis of nine reviewed approaches and the reference implementation, together with the design decisions they represent;
3) a description of the reference implementation stated precisely from its source code, including what it already enforces and what it deliberately omits;
4) an explicit research gap and a hardened reference design that addresses it; and
5) a reproducible measurement plan. No unmeasured performance claim is made.

Section II gives the technical background. Section III reviews and compares the literature. Section IV states the research gap. Section V describes the reference implementation and the proposed hardened design. Section VI presents the evaluation plan, and Section VII concludes.

II. Background

A. TCP as a Byte Stream

TCP establishes connection state before any data moves, numbers every byte so that lost segments can be retransmitted and out-of-order segments reassembled, and uses a window to stop a fast sender from overwhelming a slow receiver [1], [2]. Congestion control, introduced by Jacobson and later standardised, limits how quickly a sender injects data into the network through slow start, congestion avoidance, fast retransmit and fast recovery [4], [5]. Together these mechanisms give an application an ordered, loss-free byte stream.

They do not give it more than that. TCP does not know about users, rooms or messages, it does not persist anything across a process crash, and it does not confirm that a particular chat participant actually read a particular line. This is an instance of the end-to-end argument: functions that only the application can implement completely, such as message boundaries or delivery confirmation to a person, cannot be delegated wholesale to the lower layer [6].

B. Connection Failure and Liveness

A TCP connection can end cleanly, with an exchange of FIN segments, or it can fail through a reset, a timeout, a crashed process, a dropped interface or a network partition. An application normally learns about failure when a read returns end-of-stream, when a send or receive raises an exception, or when a liveness policy expires. TCP keep-alive is optional and off by default; the host-requirements standard requires that it be configurable and warns that a single unanswered probe is not proof that the peer is gone [3]. In practice this means that explicit quit commands and exception handling handle the common case, whereas noticing an unplugged laptop quickly needs application-level timeouts or heartbeat messages.

TABLE I
Application-Layer Framing Alternatives

Method	Advantage	Limitation
Fixed size	Trivial parsing	Wasted space; inflexible
Delimiter	Human-readable; simple for lines	Escaping and delimiter collision
Length prefix	Binary-safe; exact; variable size	Length must be validated
TLV	Extensible typed fields	Larger parser and schema
Self-describing	Rich structure (XML/JSON)	Encoding overhead; needs outer framing

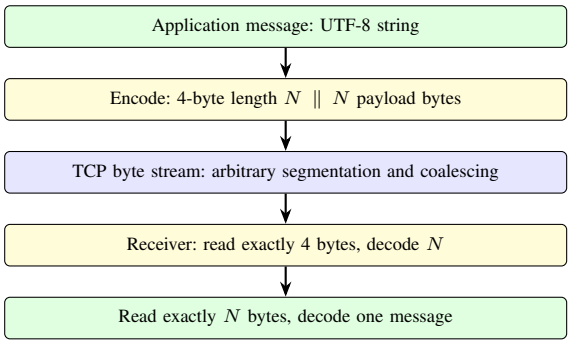


Fig. 1. Length-prefixed framing over a TCP byte stream.

C. Why Framing Is Mandatory

Because TCP segmentation is independent of application writes, a receiver cannot rely on read boundaries. Four framing strategies dominate practice, summarised in Table I. *Fixed-size* records are trivial to parse but waste space and cannot carry variable payloads. *Delimiter*-based framing, used by line-oriented protocols such as IRC, is readable but needs an escaping rule whenever the delimiter can occur inside the data [18]. *Length-prefixed* framing announces the payload size before the payload, so a receiver reads exactly that many bytes; it is binary-safe and precise, at the price of having to validate the announced length. *Self-describing* formats, such as the XML streams of XMPP, carry structure inside the data but still depend on an outer mechanism to find where one unit ends [20]. WebSocket and MQTT both use length information in binary headers [22], [23].

For a stream of UTF-8 text [10], a four-byte unsigned length in network byte order followed by the encoded payload is compact and easy to test. Python's `struct` module defines the `!` format prefix as network (big-endian) order with standard sizes, so `"!I"` is an interoperable four-byte header [34]. The receiver must loop until the requested number of bytes has accumulated or the peer closes, because a single `recv()` may return fewer bytes than asked for. Fig. 1 shows the idea, and Listing 1 shows the code used by the reference implementation.

```
MAX_MESSAGE_BYTES = 4096

def send_message(conn, message):
    payload = message.encode("utf-8")
    if len(payload) > MAX_MESSAGE_BYTES:
        raise ValueError("message is too large")
    conn.sendall(struct.pack("!I", len(payload)) + payload)

def receive_message(conn):
    header = _receive_exactly(conn, 4)
    if header is None:
        return None            # peer closed
    (length,) = struct.unpack("!I", header)
    if length == 0 or length > MAX_MESSAGE_BYTES:
        raise ValueError("invalid message length")
    payload = _receive_exactly(conn, length)
    if payload is None:
        return None
    return payload.decode("utf-8")
```

Listing 1. Framing logic of the reference implementation (`protocol.py`, condensed).

D. Concurrency Models for Servers

A chat server holds many connections open for a long time, and most of the time each one is idle. Four models are usually contrasted. *Thread-per-client* gives each connection a sequential control flow that is easy to read and debug, at the cost of one stack and one scheduling entity per client. A *thread pool* bounds the number of workers but adds queueing and makes long-lived connections awkward. An *event loop* multiplexes many connections on few threads using readiness notification, which scales further but forces the programmer to express each connection as a state machine [17], [16]. *Staged* architectures split a service into event-driven stages joined by explicit queues with dynamic resource control [12]. The debate between threads and events is old and unresolved: Ousterhout argued that threads are the wrong abstraction for most purposes, von Behren *et al.* argued that the objections are really objections to particular thread implementations, and Lee described the difficulty of reasoning about shared-memory threads [13], [14], [15].

E. Group Delivery and Topology

Delivering one message to many recipients can be done in several ways. A central server can relay by repeated unicast, sending a separate copy on each connection. Peer-to-peer designs remove the relay but introduce membership, connection-count and consistency problems. IP multicast sends one datagram to a group, but it uses datagram semantics, which moves loss recovery, ordering and membership back to the application [24]. Broker-based publish–subscribe decouples senders and receivers in space, time and synchronisation [25]. Repeated unicast is not Ethernet broadcast or IP multicast: every recipient has an independent TCP connection with its own flow-control window and failure state.

F. Security Considerations

A trusted LAN reduces deployment effort but is not a security mechanism. On a plaintext channel, any reachable host can connect, choose any name, inject messages, and read traffic. TLS 1.3 protects a client–server channel against eavesdropping, tampering and forgery and can authenticate the server with a certificate [28]. It does not authenticate individual users, and the server still sees plaintext after decryption. End-to-end secrecy between users needs a different construction, such as the Double Ratchet used by Signal, which provides forward secrecy and post-compromise security [31], [30]. Unger *et al.* give a systematic comparison of secure-messaging designs [29].

III. Literature Review

Rather than reviewing sources one by one, it is more useful to group them by the design question they answer. Table II compares the approaches discussed below, and the reference implementation, on the properties most relevant to a small LAN chat system.

Complete messaging protocols. IRC was the first widely deployed text chat protocol on TCP. Its base specification frames messages as lines ending in carriage return and line feed, limited to 512 characters, and connects servers in a network so that users on different servers can talk to each other [18], [19]. The design is easy to implement by hand, which explains its longevity, but the delimiter approach forces every reader to agree on how to handle over-long or malformed lines. XMPP takes the opposite direction: it defines a long-lived XML stream over TCP with negotiation of TLS and SASL authentication, and a separate specification adds rosters and presence for instant messaging [20], [21]. That richness is precisely what makes it heavy for a small system. MQTT 5.0 places a broker between publishers and subscribers and adds quality-of-service levels, sessions and a compact binary header [23]. WebSocket adds an HTTP-compatible opening handshake and then exchanges length-carrying frames, which makes it the natural route to browser clients [22]. All four solve interoperability and deployment problems that a command-line demonstration deliberately sidesteps.

Server-architecture research. Pai, Druschel and Zwaenepoel compared multi-process, multi-threaded, single-process event-driven and asymmetric event-driven designs in the Flash web server and showed that architecture materially changes throughput and blocking behaviour [11]. SEDA proposed decomposing a service into stages linked by bounded queues, with dynamic controllers that keep the system stable under overload [12]. Position papers on both sides of the threads-versus-events debate followed [13], [14], [15], and Krohn *et al.* argued that event-driven code can be written with a structure close to threaded code [16]. The C10K discussion collected the operating-system techniques needed to serve ten thousand simultaneous connections [17]. What these works share is that their target workloads are much larger and more heterogeneous than a room of a few terminals. They justify an architectural boundary, meaning the point at which thread-per-client stops being adequate, without proving that threads are the wrong choice below it.

Group communication and reliability. Birman and Joseph's work on virtual synchrony and the later process-group approach show what *reliable* group delivery means

TABLE II
COMPARATIVE ANALYSIS OF REVIEWED APPROACHES

Approach	Transport	Framing	Topology	Security	Persistence	Key limitation for a small teaching system
IRC [18], [19]	TCP	CRLF lines, 512-byte limit	Server network	None in base spec	None	Text-only framing; limits fixed by the spec
XMPP [20], [21]	TCP	XML stream	Federated client–server	SASL and TLS	Via extensions	Heavy parser; large specification surface
MQTT 5.0 [23]	TCP (optionally TLS)	Binary header, variable length	Broker pub/sub	Optional; credentials in CONNECT	Sessions and QoS	Needs a broker; more machinery than a chat room needs
WebSocket [22]	TCP after HTTP upgrade	Length-carrying frames; client masking	Client–server	TLS via `wss`	None	Needs an HTTP handshake and a framework
Signal / Double Ratchet [31], [30]	Any	Not specified	Relayed	End-to-end	Server-dependent	Cryptographic design, not a transport or server design
Flash [11]	TCP (web)	HTTP	Client–server	N/A	N/A	Web workload; compares process, thread and event models
SEDA [12]	TCP (services)	Application-defined	Staged pipeline	N/A	N/A	Targets far larger loads than a classroom chat
Threads vs. events [13], [14], [16]	TCP	N/A	N/A	N/A	N/A	Position papers, not chat-specific
Reliable groups [26], [27]	Reliable channels	N/A	Process groups	N/A	Varies	Guarantees far beyond a demonstration system
Reference implementation	**TCP**	**4-byte length + UTF-8**	**Central relay**	**None**	**None**	**Trusted-LAN scope; hardened design proposed in Sec. V**

when processes can fail: members agree on group views and on the order in which messages are delivered relative to membership changes [26], [27]. Eugster *et al.* survey publish–subscribe and classify it by how it decouples producers and consumers [25]. A chat server that relays by repeated unicast provides none of these guarantees. A message accepted by the server can reach some clients before another connection fails, and there is no acknowledgement set, replay or globally stable order. Distinguishing this from transport reliability is important, because TCP's reliability is easily mistaken for delivery reliability.

Secure messaging. TLS 1.3 secures the channel [28], and the secure-messaging literature separates transport security, user authentication, authorisation, and end-to-end secrecy as distinct layers [29]. Signal's Double Ratchet shows how end-to-end secrecy with forward secrecy can be combined with an untrusted relay [31], [30]. These layers are additive, and a system can adopt them in stages.

Foundational texts. Stevens' treatment of the sockets API and the standard networking textbooks remain the main sources for how a program actually uses TCP [7], [8], [9].

Read together, these groups trace a fairly clean progression: from byte-stream transport, to explicit message framing, to concurrent servers, to full messaging protocols, and finally to reliable and secure group communication. What none of them does individually is present a compact, dependency-free system in which all of these decisions interact and can be inspected and measured at once.

IV. RESEARCH GAP

Reading Table II left to right, every reviewed approach is strong in the columns it was designed for and silent elsewhere. The remainder of this section explains why closing the gaps is a matter of design, not of adding one more feature.

Framing, concurrency and shutdown are studied separately. Protocol standards specify frames, architecture papers compare schedulers, and textbooks teach sockets and threads as independent exercises. In a real chat server they interact: a handler thread blocks inside a read while another thread wants to shut the server down, a broadcast iterates a client table while another handler removes an entry, and a half-received frame is abandoned when a peer disconnects mid-message. Few sources examine these interactions in a system small enough to read in one sitting.

Sequential broadcast has a head-of-line risk. When a handler sends a message to every recipient in turn, a slow receiver whose send buffer is full can delay delivery to every recipient after it. Per-client outbound queues with writer workers, bounded queues with a disconnect-or-drop policy, or non-blocking writes isolate slow consumers. This is well known in large systems [12], but it is rarely shown in a teaching-scale implementation together with the code and tests that demonstrate the problem.

Liveness detection is left to defaults. Because TCP keep-alive is optional and off by default [3], a suspended laptop or an unplugged cable can leave a connection half-open for a long time. Explicit exit and exception cleanup cover normal

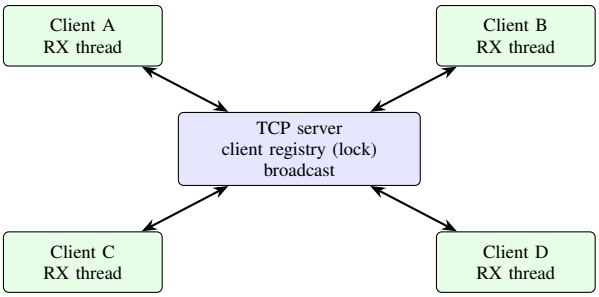


Fig. 2. Centralised architecture of the reference implementation. Each client keeps one full-duplex TCP connection; the server owns membership and relays each message by repeated unicast, one handler thread per client.

termination, but abnormal termination needs a heartbeat or timeout that is seldom included in small examples.

Security is either absent or arrives all at once. Simple demonstrations are plaintext with no identity; production systems bring TLS, accounts and end-to-end encryption together [28], [29], [31]. The intermediate steps, such as wrapping sockets with the standard `ssl` module and adding a name-uniqueness rule, are rarely presented as a staged path with a clear statement of what each stage does and does not protect.

Evaluation is anecdotal. Very few small messaging examples come with a protocol-correctness test that fragments and coalesces frames, or with a measurement method for latency and fan-out cost. Without one, no claim about when thread-per-client stops being adequate can be checked.

The reference design in Section V is built around the premise that these gaps share one root cause: transport, framing, concurrency, delivery and security are presented as separate problems, when in a chat system they are stages of one path that must agree on message size limits, lifecycle and failure handling.

V. Reference Implementation and Proposed Hardened Design

A. Reference Implementation

The case-study system is a command-line chat program written with only the Python standard library [32], [33], [34]. It is split across five source files, mapped to networking concepts in Table III. The statements below were verified against the source code.

Server. The server binds a TCP listening socket, runs an accept loop in a background thread, and starts one daemon thread per accepted connection. A handshake sends a `USERNAME` prompt and expects a name of one to 32 characters. Client state is held in a dictionary guarded by a `threading.Lock`; broadcast takes a snapshot of the recipients under the lock and then sends outside it, so a blocking send does not hold the lock. Messages are relayed to every client except the sender, and join and leave notices are broadcast in the same way. The server accepts a `quit`, `stop` or `exit` command typed into its own terminal; on that command, or on Ctrl+C, it sends a shutdown notice to each client, closes every connection, and then closes the listener.

Client. The client connects, completes the handshake, and starts a background receive thread so that incoming messages are displayed while the user is typing. The main thread reads terminal input and sends each line. The `/quit` and `/exit` commands, end-of-input and Ctrl+C all lead to a clean exit. A small helper module adds ANSI colours only when output is a terminal.

Protocol and tests. The framing in Listing 1 is shared by both programs, so there is a single definition. Frames are limited to 4096 bytes, zero-length and oversized frames are rejected, and a premature end of stream is reported as `None` rather than as an exception. The test suite has three tests: a round trip over a socket pair, the closed-connection case, and rejection of an oversized message.

Limits. Several limits follow directly from the design and are stated here so that the system's contribution is not overstated. Broadcast sends to recipients one after another with no per-client queue or timeout, so a stalled receiver can delay the ones that follow. Usernames are not required to be unique, and there is no authentication, so any host that can reach the port can join under any name. Traffic is plaintext. There is no heartbeat, so a half-open connection is found only when a send fails. There is no history, so a client that reconnects sees nothing that happened while it was away. Thread-per-client is appropriate for a classroom LAN but uses one thread per connection.

B. Proposed Hardened Design

We call the proposed design *LANChat-H* (H for hardened). It keeps the five-file structure and the dependency-free constraint, and extends the system in the four stages shown in Fig. 3. Each stage is independent and testable, and each carries forward the limit and lifecycle decisions of the one before it. Table IV summarises how the result compares with the approaches reviewed.

Stage 1 – Robustness of the existing path. Add a bounded per-client outbound queue with a writer thread, so that a slow receiver is disconnected or has messages dropped by a stated policy instead of delaying others. Add a heartbeat message and a receive timeout so that half-open connections are detected. Require unique usernames at the handshake. Extend the tests to split the four-byte header across writes, combine many frames in one write, close mid-payload and disconnect during a broadcast.

Stage 2 – Protocol structure. Replace bare strings with typed messages (chat, join, leave, system, ping) carried inside the same length-prefixed frame, and add a version byte. This makes the protocol extensible without changing the framing decision defended in Section II.

Stage 3 – Channel security and identity. Wrap both ends of the socket with the standard `ssl` module for TLS [28], [35], and add a per-user credential check and separate authorisation for administrative commands such as the server shutdown. This addresses eavesdropping, tampering and impersonation on the channel; it does not provide end-to-end secrecy, because the server still sees plaintext.

TABLE III
IMPLEMENTATION MAPPING AND HONEST SCOPE

Component	Networking concept	Role in the system	Boundary / not provided
`server.py`	Listen, accept, membership, fan-out	Central relay; one daemon thread per client; lock-protected registry; graceful shutdown	Single server; no failover; sequential broadcast; no heartbeat
`client.py`	Active open; full-duplex I/O	Foreground input plus background receive thread; clean exit paths	No reconnect; no authenticated identity; no offline delivery
`protocol.py`	Record recovery over a stream	4-byte network-order length; exact-read loop; 4096-byte limit; rejects zero-length frames	Not versioned; no message types or acknowledgements
`colors.py`	Presentation formatting	ANSI colours, disabled when not a terminal	Terminal-escape handling is not protocol security
`test_chat.py`	Protocol verification	Round trip, closed connection, oversized message	Not a measurement of scalability or security

TABLE IV
EXISTING APPROACHES VS. LANCHAT-H

Capability	Reviewed (individually)	LANChat-H
Explicit message framing	Yes	Yes
Frame-size limit enforced	Varies	Yes
Isolation of slow receivers	Large systems only	Yes
Liveness detection	Rare in small systems	Yes
Channel security (TLS)	Production systems	Yes (Stage 3)
Threaded and async comparison	No	Yes (Stage 4)
Frame-fragmentation tests	Rare	Yes
Single dependency-free artifact	No	Yes

Stage 4 – Asynchronous variant. Implement the same protocol with `asyncio` [36] and compare it with the threaded server under identical workloads, so that the point at which thread-per-client stops being adequate is measured rather than assumed. Persistent history with sequence numbers and acknowledgements can be studied as a further extension.

VI. EVALUATION PLAN

No valid performance conclusion can be drawn without measurements on a specified machine, Python version, LAN, payload distribution and client count, so none is reported here. A reproducible experiment should vary the number of clients C, the payload size S and the offered message rate R. For a sender timestamp t_s and a recipient timestamp $t_{r,i}$, the latency seen by recipient i is

$$L_i = t_{r,i} - t_s. \tag{1}$$

A broadcast is summarised by the median, 95th percentile and maximum of L_i, and by the completion spread

$$\Delta = \max_i L_i - \min_i L_i. \tag{2}$$

Application throughput is the number of fully reconstructed frames per second, not raw TCP segments.

Fan-out cost model. The head-of-line risk described in Section IV can be stated precisely. Let the server relay a message to N recipients in a fixed order, and let s_j be the time the blocking send to recipient j takes. With sequential sends, recipient k receives the message after

$$D_k = \sum_{j=1}^{k} s_j, \qquad T_{\text{bcast}} = D_N. \tag{3}$$

TABLE V
SUGGESTED MEASUREMENTS

Metric	Method	Purpose
Latency	Monotonic timestamps at sender and receivers	End-to-end responsiveness
Throughput	Frames and bytes per second	Capacity under load
Fan-out cost	Delivery time vs. number of recipients	Broadcast scaling
CPU / memory	Process sampling under steady load	Thread and queue overhead
Failure recovery	Time from disconnect to cleanup notice	Fault responsiveness
Correctness	Fragmented and coalesced frames in tests	Verify exact-read framing

If recipient m stalls for an extra time W because its send buffer is full, every later recipient is delayed by at least the same amount:

$$D_k \;\geq\; W + \sum_{j=1}^{k} s_j \qquad \text{for all } k \geq m. \tag{4}$$

With a bounded per-client queue and a writer thread (Stage 1), the server's send to each recipient reduces to an enqueue with cost $q_j \ll W$, so D_k no longer depends on W for $k \neq m$. Table V lists the planned measurements; D_k is measured directly, with and without an artificially slowed receiver, to confirm the effect.

The harness should generate automated clients, start trials together, warm the system up, run repeated fixed-duration trials, and report dispersion rather than a single best run. Adversarial cases should split the four-byte header across writes, combine many frames in one write, close in the middle of a payload, send zero-length and oversized frames, and disconnect during a broadcast, so that protocol correctness is evaluated separately from speed.

VII. CONCLUSION

Taken together, the literature on TCP-based messaging tells a story of solid but scattered progress. Transport standards define a reliable byte stream, framing conventions recover messages from it, server-architecture research explains when threads give way to events, messaging protocols fix particular

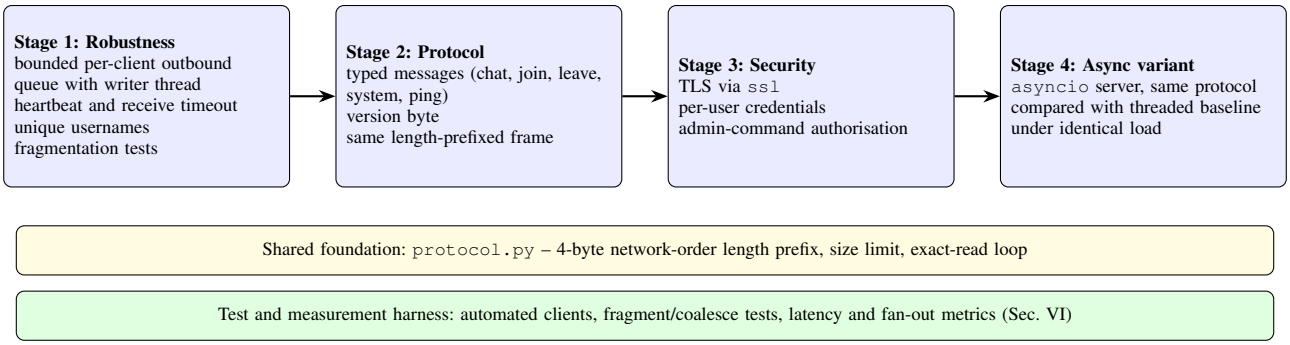


Fig. 3. Overall architecture of the proposed LANChat-H design. Four independent stages extend the existing threaded server while sharing one framing definition and one test and measurement harness.

design points, and group-communication and security work describe guarantees well beyond a demonstration system. What this survey's comparison and gap analysis both point to is that no single small artifact walks through all of these decisions together in a form that can be read, tested and measured.

The four-byte length prefix with exact-read logic is a sound choice for a stream of UTF-8 text, thread-per-client is understandable and sufficient at classroom scale, and a central relay simplifies membership while creating a single point of failure. The reference implementation makes these choices explicitly and states what it omits. The hardened design proposed here carries the same limit and lifecycle decisions through bounded queues, heartbeats, typed messages, TLS and an asynchronous variant, so that each stage can be verified before the next is added. The next phase is to implement the four stages, run the measurement plan of Section VI on a specified LAN, and report the results, including where the threaded design stops being adequate.

REFERENCES

[1] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981.
[2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022.
[3] R. Braden, Ed., "Requirements for Internet Hosts – Communication Layers," IETF, RFC 1122, Oct. 1989.
[4] V. Jacobson, "Congestion avoidance and control," in *Proc. ACM SIGCOMM*, 1988, pp. 314–329.
[5] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," IETF, RFC 5681, Sep. 2009.
[6] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Trans. Comput. Syst.*, vol. 2, no. 4, pp. 277–288, 1984.
[7] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Vol. 1: The Sockets Networking API*, 3rd ed. Addison-Wesley, 2003.
[8] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson, 2011.
[9] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.
[10] F. Yergeau, "UTF-8, a transformation format of ISO 10646," IETF, RFC 3629, Nov. 2003.
[11] V. S. Pai, P. Druschel, and W. Zwaenepoel, "Flash: An efficient and portable Web server," in *Proc. USENIX Annu. Tech. Conf.*, 1999.
[12] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in *Proc. ACM SOSP*, 2001, pp. 230–243.
[13] J. K. Ousterhout, "Why threads are a bad idea (for most purposes)," invited talk, USENIX Winter Tech. Conf., 1996.
[14] R. von Behren, J. Condit, and E. Brewer, "Why events are a bad idea (for high-concurrency servers)," in *Proc. HotOS IX*, 2003.
[15] E. A. Lee, "The problem with threads," *Computer*, vol. 39, no. 5, pp. 33–42, May 2006.
[16] M. Krohn, E. Kohler, and M. F. Kaashoek, "Events can make sense," in *Proc. USENIX Annu. Tech. Conf.*, 2007.
[17] D. Kegel, "The C10K problem," 1999. [Online]. Available: http://www.kegel.com/c10k.html
[18] J. Oikarinen and D. Reed, "Internet Relay Chat protocol," IETF, RFC 1459, May 1993.
[19] C. Kalt, "Internet Relay Chat: Client protocol," IETF, RFC 2812, Apr. 2000.
[20] P. Saint-Andre, "Extensible Messaging and Presence Protocol (XMPP): Core," IETF, RFC 6120, Mar. 2011.
[21] P. Saint-Andre, "Extensible Messaging and Presence Protocol (XMPP): Instant messaging and presence," IETF, RFC 6121, Mar. 2011.
[22] I. Fette and A. Melnikov, "The WebSocket protocol," IETF, RFC 6455, Dec. 2011.
[23] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019.
[24] S. Deering, "Host extensions for IP multicasting," IETF, RFC 1112, Aug. 1989.
[25] P. Th. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, Jun. 2003.
[26] K. P. Birman and T. A. Joseph, "Reliable communication in the presence of failures," *ACM Trans. Comput. Syst.*, vol. 5, no. 1, pp. 47–76, 1987.
[27] K. P. Birman, "The process group approach to reliable distributed computing," *Commun. ACM*, vol. 36, no. 12, pp. 37–53, Dec. 1993.
[28] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," IETF, RFC 8446, Aug. 2018.
[29] N. Unger *et al.*, "SoK: Secure messaging," in *Proc. IEEE Symp. Security and Privacy*, 2015, pp. 232–249.
[30] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the Signal messaging protocol," in *Proc. IEEE Eur. Symp. Security and Privacy*, 2017, pp. 451–466.
[31] M. Marlinspike and T. Perrin, "The Double Ratchet algorithm," Signal, Rev. 1, Nov. 2016.
[32] Python Software Foundation, "socket," *Python 3 Documentation*. https://docs.python.org/3/library/socket.html
[33] Python Software Foundation, "threading," *Python 3 Documentation*. https://docs.python.org/3/library/threading.html
[34] Python Software Foundation, "struct," *Python 3 Documentation*. https://docs.python.org/3/library/struct.html
[35] Python Software Foundation, "ssl," *Python 3 Documentation*. https://docs.python.org/3/library/ssl.html
[36] Python Software Foundation, "asyncio," *Python 3 Documentation*. https://docs.python.org/3/library/asyncio.html